Jaca

Jaca é um interpretador de uma linguagem imperativa simples, desenvolvido em Java. O projeto usa JFlex para gerar o analisador léxico e Java CUP para gerar o parser. O parser constrói uma árvore sintática abstrata (AST), que é então executada por um interpretador baseado no padrão Visitor.

Requisitos

- JDK 17 ou superior
- Maven

Compilação

Na raiz do projeto, execute:

```
mvn clean package
```

O Maven gera o scanner e o parser, compila o projeto e empacota as dependências no arquivo:

```
target/jacacc-1.0-jar-with-dependencies.jar
```

Execução

Para interpretar um arquivo:

```
java -jar target/jacacc-1.0-jar-with-dependencies.jar caminho/para/programa.input
```

Por exemplo, para executar o programa de demonstração incluído:

```
java -jar target/jacacc-1.0-jar-with-dependencies.jar teste.input
```

Sem argumentos, o programa inicia um REPL:

```
java -jar target/jacacc-1.0-jar-with-dependencies.jar
```

No REPL, digite um ou mais comandos e pressione Enter em uma linha vazia para executar o bloco. Digite `exit` para encerrar. As variáveis e seus valores permanecem disponíveis entre os blocos executados na mesma sessão.

Linguagem

Comentários e identificadores

- Comentários começam com `#` e vão até o fim da linha.
- Identificadores começam com uma letra ASCII ou `_`; os caracteres seguintes podem ser letras, números ou `_`.

- Um identificador pode ter até 32 caracteres.
- Palavras reservadas: `if`, `then`, `else`, `while`, `print` e `read`.

Números e valores

Literais numéricos são armazenados como números de ponto flutuante (`Double`). São aceitas formas inteiras, decimais e notação científica, por exemplo: `12`, `3.5` e `6.02e-3`.

Durante a execução, os valores podem ser numéricos ou booleanos. Comparações produzem booleanos. Não há literais booleanos na sintaxe da linguagem; `read` aceita `true` ou `false` como entrada.

Comandos

Todo comando de atribuição, impressão ou leitura termina com `;`. Blocos são delimitados por chaves e podem conter comandos sequenciais.

```
contador = 1;
print(contador);
read(valor);

{
    contador = contador + 1;
    print(contador);
}
```

Comandos disponíveis:

Forma	Comportamento
`identificador = expressão;`	Avalia a expressão e atribui o resultado à variável.
`print(expressão);`	Avalia e imprime o resultado.
`read(identificador);`	Lê um número ou booleano da entrada padrão e armazena na variável.
`if (expressão) then { ... }`	Executa o bloco se a expressão resultar em booleano verdadeiro.
`if (expressão) then { ... } else { ... }`	Escolhe um dos blocos conforme a condição.
`while (expressão) { ... }`	Repete o bloco enquanto a condição resultar em booleano verdadeiro.
`{ ... }`	Agrupa uma sequência de comandos.
`;`	Comando vazio, sem efeito.

Condições de `if` e `while` devem resultar em booleano. Por exemplo:

```
idade = 20;
if (idade >= 18) then {
    print(1);
} else {
    print(0);
}
```

Expressões e operadores

Operadores	Operação	Tipos aceitos
`-` (unário)	Negação numérica	Número
`*`, `/`, `%`	Multiplicação, divisão e resto	Dois números
`+`, `-`	Adição e subtração	Dois números
`==`, `!=`	Igualdade e diferença	Dois números ou dois booleanos do mesmo tipo
`>`, `<`, `>=`, `<=`	Comparações de magnitude	Dois números

Da maior para a menor precedência, a ordem é: menos unário; `*`, `/`, `%`; `+`, `-`; e os operadores relacionais e de igualdade, que compartilham o mesmo nível de precedência e são associativos à esquerda. Parênteses podem ser usados para agrupar expressões.

Não há conversão implícita entre números e booleanos. Divisão por zero, uso de variável não inicializada e incompatibilidade de tipos geram erros durante a execução.

Exemplo completo

```
# Imprime os números de 1 a 5
i = 1;
while (i <= 5) {
    print(i);
    i = i + 1;
}

read(resposta);
if (resposta == 1) then {
    print(100);
} else {
    print(0);
}
```

No exemplo, `read` solicita um valor para `resposta` na entrada padrão. A comparação `resposta == 1` resulta em booleano, apropriado para a condição do `if`.

Organização do projeto

```
src/main/
├── cup/parser.cup                # Gramática e ações que constroem a AST
├── jflex/scanner.jflex           # Regras léxicas
└── java/parser/
    ├── Main.java                 # Entrada do programa, execução de arquivo e REPL
    └── ast/
        ├── Interpreter.java      # Avaliação e execução dos nós da AST
        ├── ASTNode.java          # Tipo base dos nós
        ├── ASTVisitor.java       # Interface Visitor
        ├── Value.java            # Valores numéricos e booleanos
        ├── expression/           # Nós de expressões
        └── statement/            # Nós de comandos
```

Os arquivos Java do scanner e do parser são gerados pelo Maven em `target/generated-sources/`; edite as especificações em `src/main/jflex/scanner.jflex` e `src/main/cup/parser.cup`, não os arquivos gerados.